

Software Requirements Specification (SRS)

Project: Bank Management System Team: Team 6 Version: 1.1 Date: 03-09-2026 Status: Draft

Team Members

USN	Name
PES2UG24CS167	Ganavi Purushothama
PES2UG24CS152	Devopam Pal
PES2UG24CS150	Deepthi V
PES2UG24CS161	Divyanshi Verma

Revision History

Version	Date	Author	Change Summary	Approval
1.0	03-09-2026	Team 6	Initial SRS draft	Completed
1.1	05-09-2026	Team 6	Transitioned to web-based architecture, integrated PostgreSQL, and added networking interfaces.	Pending

Table of Contents

- 1. [Introduction](#)
- 2. [Overall Description](#)
- 3. [External Interface Requirements](#)
- 4. [System Features \(Detailed\)](#)
- 5. [Non-Functional Requirements](#)
- 6. [Quality Attributes & Acceptance Tests](#)
- 7. [System Models and Diagrams](#)
- 8. [Requirements Traceability Matrix \(RTM\)](#)

1. Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for the **Bank Management System**, a web-based application with a core backend engineered in C/C++. It describes the functional and non-functional requirements of a system designed to handle core banking operations such as account creation, deposits, withdrawals, and balance inquiry. This document is intended for the development team, course instructors, and evaluators.

1.2 Scope

The Bank Management System will allow a bank to manage customer accounts and perform basic banking transactions through an interactive web interface. The system covers:

- Creation and management of customer accounts
- Deposit and withdrawal of funds
- Balance inquiry
- Persistent storage of account data using a PostgreSQL database
- Basic authentication for account access
- Secure online client-server communication

The system does **not** cover: multi-branch networking or integration with external payment gateways. It is intended as an academic/learning project showcasing a full-stack C/C++ web implementation.

1.3 Audience

Developers (Team 6), Course Instructor/Evaluator, QA reviewers (peer teams).

1.4 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
GUI	Graphical User Interface
HTTP/HTTPS	Hypertext Transfer Protocol (Secure)
PIN	Personal Identification Number
DB	Database
RTM	Requirements Traceability Matrix

2. Overall Description

2.1 Product Perspective

The Bank Management System is a web-based application. It utilizes a core backend developed in C/C++ to process business logic and banking operations. It interacts with clients through a responsive web interface and connects to a robust PostgreSQL database to securely persist account records across sessions.

2.2 Major Product Functions

- Create a new customer account
- Authenticate a user (account number + PIN)
- Deposit money into an account
- Withdraw money from an account
- Check account balance
- View a mini-statement / transaction history
- Update account holder details
- Close/delete an account
- Admin functions: view all accounts, search account by number/name

2.3 User Roles and Characteristics

Role	Description
Customer	End user who performs deposit, withdrawal, and balance inquiry on their own account via the web portal. Assumed to have basic computer literacy.
Bank Admin/Employee	Manages accounts, can create/close accounts and view all records through an administrative web dashboard. Assumed to be trained on the system.

2.4 Operating Environment

- **Client:** Modern Web Browsers (e.g., Chrome, Firefox, Safari, Edge) on desktop or mobile devices.
- **Server OS:** Runs on Windows/Linux desktop or lab machines.
- **Backend Environment:** Compiled using GCC/G++ (C/C++11 or later).
- **Database:** PostgreSQL server for robust relational data storage.

2.5 Constraints

- Core backend logic must strictly utilize C and C++.
- Must be implemented as an online application serving web-based clients.
- Requires concurrent session handling to support online use.

2.6 Assumptions and Dependencies

- Users have valid, unique account numbers assigned at account creation.
- The host server machine has a working C/C++ compiler, network access, and an active PostgreSQL service.
- Database records are not tampered with outside the application API.

3. External Interface Requirements

3.1 User Interfaces

- Interactive web-based Graphical User Interface (GUI) accessible via standard browsers.
- Clear, responsive web forms for actions like Create Account, Deposit, Withdraw, and Balance Inquiry.
- Formatted, tabular HTML views for statements and account lists.
- Appropriate input validation messages presented directly on the UI.

3.2 Hardware Interfaces

- Server requires network interface cards (NIC) for handling web traffic.
- Local or networked disk storage for the PostgreSQL database instance.

3.3 Software Interfaces

- **Database API:** PostgreSQL C/C++ interface (e.g., `libpq` or similar wrapper) for executing SQL queries and managing data persistence.
- **Web Framework/Server:** A C/C++ web deployment framework or custom server engine for handling HTTP routing and client requests.

3.4 Communications Interfaces

- Client-server communication operates over HTTP/HTTPS protocols.
 - System requires extensive networking capabilities to handle concurrent web traffic.
 - TCP/IP for stable database connectivity between the backend application and the PostgreSQL server.
 - Focus on secure transmission protocols to protect financial transaction data.
-

4. System Features (Detailed)

Each requirement includes acceptance criteria and a reference test case. IDs follow **BMS-F-###**.

4.1 Account Management

Description: Create, update, view, and close customer accounts.

Req ID	Requirement (shall...)	Type	Priority	Source	Acceptance Criteria / Test Ref	Comments/Dependencies
BMS-F-001	The system shall allow an admin to create a new account by capturing name, initial deposit, and generating a unique account number.	Functional	High	Business	AC-001: New account created with unique ID and correct initial balance. Test: TC-Acc-01	Requires DB write
BMS-F-002	The system shall assign a unique account number automatically and prevent duplicates.	Functional	High	Business	AC-002: No two accounts share an account number. Test: TC-Acc-02	—
BMS-F-003	The system shall require a minimum initial deposit to open an account.	Functional	Medium	Business	AC-003: Account creation rejected if deposit < minimum. Test: TC-Acc-03	Minimum configurable
BMS-F-004	The system shall allow an admin to view details of any account by account number.	Functional	Medium	Admin	AC-004: Correct account details displayed. Test: TC-Acc-04	—
BMS-F-005	The system shall allow an admin to close/delete an account.	Functional	Medium	Business	AC-005: Closed account no longer accessible; balance settled. Test: TC-Acc-05	—
BMS-F-006	The system shall allow updating of customer contact details (name/phone/address).	Functional	Low	Customer	AC-006: Updated details persist and display correctly. Test: TC-Acc-06	—

4.2 Authentication

Description: Verify identity of a user before allowing account operations.

Req ID	Requirement (shall...)	Type	Priority	Source	Acceptance Criteria / Test Ref	Comments/Dependencies
BMS-F-007	The system shall authenticate a customer using account number and PIN/password before allowing deposit/withdrawal/balance inquiry.	Functional	High	Security	AC-007: Access denied on incorrect credentials. Test: TC-Auth-01	—
BMS-F-008	The system shall mask PIN/password entry on standard web password fields.	Functional	Medium	Security	AC-008: PIN characters masked as dots/asterisks in the UI. Test: TC-Auth-02	UI standard
BMS-F-009	The system shall lock further attempts after 3 consecutive failed PIN entries within a session.	Functional	High	Security	AC-009: 4th consecutive failed attempt blocks further tries and logs the event. Test: TC-Auth-03	—

4.3 Deposit

Description: Allow a customer to deposit funds into their own account.

Req ID	Requirement (shall...)	Type	Priority	Source	Acceptance Criteria / Test Ref	Comments/Dependencies
BMS-F-010	The system shall allow a customer to deposit a positive amount into their authenticated account.	Functional	High	Business	AC-010: Balance increases exactly by deposit amount. Test: TC-Dep-01	—
BMS-F-011	The system shall reject deposit amounts that are zero, negative, or non-numeric.	Functional	High	Business	AC-011: Invalid input rejected with error message; balance unchanged. Test: TC-Dep-02	—
BMS-F-012	The system shall record every deposit as a transaction entry with timestamp and amount.	Functional	Medium	Audit	AC-012: Transaction log contains new entry after deposit. Test: TC-Dep-03	Depends on 4.6

4.4 Withdrawal

Description: Allow a customer to withdraw funds from their own account, subject to balance rules.

Req ID	Requirement (shall...)	Type	Priority	Source	Acceptance Criteria / Test Ref	Comments/Dependencies
BMS-F-013	The system shall allow a customer to withdraw an amount not exceeding their current balance.	Functional	High	Business	AC-013: Withdrawal succeeds only if amount $\leq$ balance. Test: TC-Wd-01	This step has to be taken with utmost care and with proper authorization
BMS-F-014	The system shall reject withdrawal requests that would result in a negative balance below the minimum balance requirement.	Functional	High	Business	AC-014: Withdrawal rejected with clear message when insufficient funds. Test: TC-Wd-02	This step has to be taken with utmost care and with proper authorization
BMS-F-015	The system shall record every withdrawal as a transaction entry with timestamp and amount.	Functional	Medium	Audit	AC-015: Transaction log contains new entry after withdrawal. Test: TC-Wd-03	Depends on 4.6

4.5 Balance Inquiry

Description: Allow a customer to view their current account balance.

Req ID	Requirement (shall...)	Type	Priority	Source	Acceptance Criteria / Test Ref	Comments/Dependencies
BMS-F-016	The system shall display the current balance of an authenticated customer's account on request.	Functional	High	Business	AC-016: Displayed balance matches stored DB balance exactly. Test: TC-Bal-01	—

4.6 Transaction History / Mini-Statement

Description: Maintain and display a record of recent transactions.

Req ID	Requirement (shall...)	Type	Priority	Source	Acceptance Criteria / Test Ref	Comments/Dependencies
BMS-F-017	The system shall maintain a persistent database log of all deposit and withdrawal transactions per account.	Functional	Medium	Audit	AC-017: DB table contains all past transactions reliably. Test: TC-Hist-01	PostgreSQL persistence
BMS-F-018	The system shall display the last N transactions (mini-statement) for an authenticated account on request.	Functional	Medium	Customer	AC-018: Correct chronological list of recent transactions shown on UI. Test: TC-Hist-02	—

4.7 Data Persistence

Description: Ensure account and transaction data is securely stored.

Req ID	Requirement (shall...)	Type	Priority	Source	Acceptance Criteria / Test Ref	Comments/Dependencies
BMS-F-019	The system shall save all account data to the PostgreSQL database in real-time upon transaction completion.	Functional	High	Reliability	AC-019: Data committed successfully, maintaining ACID properties. Test: TC-Persist-01	—
BMS-F-020	The system shall retrieve real-time account data from the DB for client sessions.	Functional	High	Reliability	AC-020: Previously created accounts are available immediately across web sessions. Test: TC-Persist-02	—

5. Non-Functional Requirements (Detailed)

IDs follow **BMS-NF-###**.

Req ID	Requirement	Category	Priority	Acceptance Criteria / Measurement
BMS-NF-001	Any single transaction (deposit/withdrawal/balance inquiry) shall complete and display a result on the web interface within 2 seconds under normal load.	Performance	High	Manual timing test; Test: TC-Perf-01
BMS-NF-002	The database shall enforce transaction integrity to prevent silent data loss during concurrent access or unexpected closures.	Reliability	High	Concurrency/ACID tests; Test: TC-Rel-01
BMS-NF-003	The system shall validate all user inputs (web forms, amounts, account numbers) both client-side and server-side to prevent crashes or SQL injection.	Robustness	High	Fuzz/injection test suite passes; Test: TC-Rob-01
BMS-NF-004	The Web UI forms shall be intuitive enough that a first-time user can complete a deposit within 3 attempts without external help.	Usability	Medium	Informal usability walkthrough; Test: TC-UX-01
BMS-NF-005	The C/C++ backend codebase shall be modular (separate files for routing, business logic, and DB I/O) to support maintainability.	Maintainability	Medium	Code review checklist; Test: TC-Maint-01
BMS-NF-006	The web server and database architecture shall support at least 500 customer accounts and concurrent sessions without noticeable performance degradation.	Scalability	Low	Load test with 500 dummy accounts/sessions; Test: TC-Scale-01

5.1 Security

5.1.1 Security Objectives

1. **Confidentiality of credentials:** PINs/Passwords must not be stored or displayed in plain text during operation or in the database.
2. **Integrity of financial data:** Account balances and transaction records must not be alterable except through validated API operations.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance Criteria / Test Ref
BMS-SR-001	The system shall require account number + PIN authentication before any account-specific operation over HTTP/HTTPS.	Security	High	AC: No transaction possible without successful auth token/session. Test: TC-Sec-01
BMS-SR-002	The system shall store PINs as salted hashes in the PostgreSQL database rather than plain text.	Security	High	AC: DB inspection shows no plain-text PINs. Test: TC-Sec-02
BMS-SR-003	The system shall lock an account's access after 3 consecutive failed PIN attempts and require admin intervention.	Security	High	AC: Locked account cannot authenticate until reset. Test: TC-Sec-03
BMS-SR-004	The system shall restrict admin-only functions via role-based access control (RBAC) to an authenticated admin session.	Security	High	AC: Customer session cannot access admin UI routes. Test: TC-Sec-04
BMS-SR-005	The system shall log security-relevant network events (failed logins, account lockouts, account deletions) with timestamps.	Security	Medium	AC: Security log contains expected entries after test scenarios. Test: TC-Sec-05

6. Quality Attributes & Acceptance Tests

- **Exit criteria for acceptance:** All high-priority functional requirements implemented and verified; no critical NFR failures; RTM shows all test cases passed or explicitly waived.
 - **Acceptance test suites:** Account Creation, Authentication, Deposit, Withdrawal, Balance Inquiry, Transaction History, Persistence, Security, and Network Load.
-

7. Requirements Traceability Matrix (RTM)

Req ID	Requirement (short)	Section Ref	Module	Test Case(s)	Status (N/P/A)
BMS-F-001	Create account	4.1	AccountModule	TC-Acc-01	N
BMS-F-007	Authenticate user	4.2	AuthModule	TC-Auth-01	N
BMS-F-010	Deposit funds	4.3	TransactionModule	TC-Dep-01	N
BMS-F-013	Withdraw funds	4.4	TransactionModule	TC-Wd-01, TC-Wd-02	N
BMS-F-016	Balance inquiry	4.5	AccountModule	TC-Bal-01	N
BMS-F-019	Persist data to DB	4.7	DatabaseModule	TC-Persist-01	N
BMS-NF-001	Web Response time target	5	All	TC-Perf-01	N
BMS-SR-002	Hash PINs in DB	5.1.2	AuthModule	TC-Sec-02	N

(P = Pass, A = Actioned/Fail — update as testing progresses.)*